Installation, Prerequisites, Dependencies and Assumptions

This chapter explains how to install, configure, start, and use the NightOUT application.

NightOUT is an academic web application composed of a Java Spring Boot backend and a React TypeScript frontend. The backend provides the REST APIs and manages the application data, while the frontend provides the graphical interface used by Normal Users, Venue Managers, and PR Managers.

The following sections describe the software requirements, the installation procedure, the main project dependencies, and the assumptions under which the application operates. Instructions for verifying the installation and solving the most common execution problems are also provided.

In the repository, the executable source code is located inside the `IMPLEMENTATION` directory:

- `IMPLEMENTATION/backend` contains the Spring Boot backend;
- `IMPLEMENTATION/frontend` contains the React frontend.

Both components must be running at the same time to use all the functionalities of the application.

Prerequisites

Before installing and running NightOUT, the following software must be available on the computer.

Software	Required version	Purpose
Git	Current supported version	Used to clone the project repository from GitHub. On Windows, Git Bash can also be used to execute the Maven Wrapper.
Java Development Kit	JDK 21	Required to compile and run the Spring Boot backend. A Java Runtime Environment alone is not sufficient.

Node.js	Version 20.19 or version 22.12 and newer	Required to run the frontend development environment and Vite.
npm	Version included with Node.js	Used to install the frontend dependencies and run the available frontend commands.
Modern web browser	Current version	Used to access and test the NightOUT web interface.
Code editor or IDE	Optional	Useful for inspecting or modifying the source code, but not required to run the application.

Apache Maven does not need to be installed separately. The backend includes the Maven Wrapper files `mvnw`, `mvnw.cmd`, and the `.mvn` directory. The wrapper automatically downloads and uses the Maven version configured by the project.

An internet connection is required during the first installation because Maven and npm must download the project dependencies.

Recommended Software

The following software can be used to satisfy the prerequisites:

- Git for Windows;
- Eclipse Temurin JDK 21 or another compatible JDK 21 distribution;
- a supported version of Node.js;
- Microsoft Edge, Google Chrome, Mozilla Firefox, or another modern browser;
- Visual Studio Code, IntelliJ IDEA, or another preferred development environment.

Verify the Installed Tools

After installing the required software, open a new PowerShell terminal and run:

```
git --version

java -version

javac -version

node --version
```

npm.cmd --version

The Java commands must report major version 21. Both `java` and `javac` must be available because the backend requires the complete Java Development Kit.

The Node.js version must satisfy one of the following conditions:

- version 20.19 or newer within the Node.js 20 release line;
- version 22.12 or newer;
- a later compatible version.

On Windows, the command `npm.cmd` is used in this guide because some PowerShell configurations block the `npm.ps1` script. When the `npm` command works correctly, `npm` and `npm.cmd` can be used equivalently.

For example:

npm --version

and

npm.cmd --version

should return the same installed npm version.

If a command is not recognized, close the current terminal and open a new one. If the problem remains, verify that the corresponding software has been installed correctly and added to the system `PATH`.

Installation and Execution

This section describes how to obtain the NightOUT source code, start the backend and frontend components, and verify that the application is working correctly.

The installation procedure is presented primarily for Windows using PowerShell. Equivalent commands for macOS and Linux are provided in a later subsection.

Clone or Download the Repository

The recommended method for obtaining the project is to clone the GitHub repository using Git.

Open PowerShell and move to the directory in which the project should be stored:

cd C:\path\to\projects

Then clone the NightOUT repository:

git clone
https://github.com/riccardomasarin/SE4A_RiccardoMasarin_AlessandroForlani_ManuelManucci.git

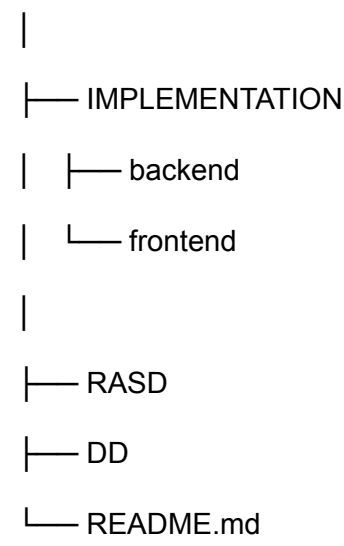
Enter the newly created project directory:

cd .\SE4A_RiccardoMasarin_AlessandroForlani_ManuelManucci

The current directory is now the repository root. It contains the main project documentation and the `IMPLEMENTATION` directory, which includes the executable application.

The relevant structure is:

SE4A_RiccardoMasarin_AlessandroForlani_ManuelManucci



The backend source code is located in:

IMPLEMENTATION/backend

The frontend source code is located in:

IMPLEMENTATION/frontend

All the commands described in the following subsections assume that the user starts from the repository root.

Alternative: Download the ZIP Archive

The project can also be downloaded directly from GitHub as a ZIP archive.

To use this method:

1. open the project repository on GitHub;
2. select the **Code** button;
3. select **Download ZIP**;
4. extract the downloaded archive;
5. open PowerShell inside the extracted project folder.

After extraction, verify that the selected directory directly contains the `IMPLEMENTATION` folder. The backend and frontend commands must be executed inside their corresponding subdirectories.

Cloning the repository with Git is preferable because it allows the project to be updated more easily using Git commands. Downloading the ZIP archive is sufficient when the objective is only to install and demonstrate the application.

Backend Setup

The NightOUT backend is implemented using Java Spring Boot and must be started before the frontend.

From the repository root, enter the backend directory:

cd .\IMPLEMENTATION\backend

Before starting the application, verify that Java 21 is correctly installed:

java -version

javac -version

Both commands must report version 21.

Start the Backend with the Maven Wrapper

The project includes the Maven Wrapper, so Apache Maven does not need to be installed globally.

On Windows, the intended command is:

.\mvnw.cmd spring-boot:run

The wrapper automatically downloads the Maven version required by the project and all the necessary backend dependencies.

The first execution may take longer because the dependencies must be downloaded from the internet.

When the startup process is successful, the terminal displays the Spring Boot banner and a final message similar to:

Started BackendApplication

The terminal must remain open while the application is being used.

To stop the backend, press:

Ctrl+C

Windows Maven Wrapper Workaround

In some Windows PowerShell environments, the `mvnw.cmd` script may fail before Maven starts and display an error related to `Target[0]` or the message:

Cannot start maven from wrapper

This error is caused by the Windows wrapper script and does not indicate a problem with Java or the NightOUT application.

A verified workaround is to execute the POSIX Maven Wrapper through Git Bash.

From PowerShell, while located in `IMPLEMENTATION\backend`, run:

& "$env:ProgramFiles\Git\bin\bash.exe" -lc './mvnw spring-boot:run'

Alternatively, open Git Bash, move to the backend directory, and run:

./mvnw spring-boot:run

This procedure still uses the Maven Wrapper included in the repository. Therefore, a global Maven installation is not required.

Verify the Backend

By default, the backend runs on port `8080`.

After the startup process is complete, open the following addresses in a web browser:

Address	Expected result
`http://localhost:8080/`	The message `NightOUT backend is running`
`http://localhost:8080/api/health`	The message `OK`
`http://localhost:8080/h2-console`	The H2 database login page

The backend can also be verified from another PowerShell terminal:

Invoke-RestMethod http://localhost:8080/

Invoke-RestMethod http://localhost:8080/api/health

If the backend is running correctly, the commands return:

NightOUT backend is running

OK

H2 Database Configuration

NightOUT uses an in-memory H2 database. No external database server needs to be installed or configured.

The local database settings are:

Property	Value
JDBC URL	`jdbc:h2:mem:nightoutdb`
Username	`sa`
Password	Empty
Schema policy	`create-drop`

To access the H2 console, open:

http://localhost:8080/h2-console

Enter the JDBC URL and username shown above and leave the password field empty.

The application automatically loads demo users, venues, events, tickets, pregames, promotions, notifications, friendships, and other records when the backend starts.

Because the database is stored in memory, any changes performed while using the application are temporary. When the backend is stopped, the data are deleted. The original demo dataset is created again the next time the backend starts.

Frontend Setup

The NightOUT frontend is implemented using React, TypeScript, and Vite. It must be started in a separate terminal while the backend remains running.

Open a new PowerShell terminal and move to the repository root. Then enter the frontend directory:

cd .\IMPLEMENTATION\frontend

Verify that Node.js and npm are correctly installed:

node --version

npm.cmd --version

The installed Node.js version must be compatible with the version required by Vite.

Install the Frontend Dependencies

The repository includes a `package-lock.json` file, which records the exact versions of the installed dependencies.

To install the frontend dependencies, run:

npm.cmd ci

The `npm ci` command is recommended because it installs the exact dependency versions recorded in `package-lock.json`, providing a reproducible installation.

The first installation requires an internet connection and may take some time while npm downloads the required packages.

The command creates the local `node_modules` directory inside the frontend folder.

The alternative command:

npm.cmd install

should be used only when intentionally updating or resolving the dependencies declared in `package.json`. Unlike `npm ci`, it may modify the `package-lock.json` file.

Start the Frontend

After the dependencies have been installed, start the Vite development server:

npm.cmd run dev

When the startup is successful, the terminal displays a message containing the local application address:

http://localhost:5173

Open this address in a modern web browser:

http://localhost:5173

The NightOUT login page should appear.

The frontend terminal must remain open while the application is being used. To stop the frontend server, press:

Ctrl+C

If the PowerShell environment allows the npm script shim, the shorter equivalent commands can also be used:

npm ci

npm run dev

On Windows, this guide uses `npm.cmd` because some PowerShell execution policies may block the `npm.ps1` script.

Frontend and Backend Connection

The frontend communicates with the backend REST API at:

http://localhost:8080/api

For this reason, the backend must already be running before the application workflows are tested.

The expected local configuration is:

Component	Address
Frontend	`http://localhost:5173`

Backend	`http://localhost:8080`
Backend API	`http://localhost:8080/api`

The frontend must be opened using the exact address `http://localhost:5173`.

If port 5173 is already occupied, Vite may automatically select another port, such as 5174. However, the current backend CORS configuration only accepts requests from `http://localhost:5173`. Therefore, the application may open on the alternative port but fail to communicate with the backend.

In this case, stop the process that is using port 5173 and restart the frontend.

Build Verification

The frontend can be compiled into a production bundle using:

npm.cmd run build

A successful build confirms that the TypeScript source code can be compiled and that Vite can generate the frontend bundle.

The project also includes a lint command:

npm.cmd run lint

The lint command performs static code-quality checks using ESLint.

In the current project version, the production build succeeds, while the lint command reports existing warnings and errors, mainly related to React Hooks rules. These lint results do not prevent the application from running with:

npm.cmd run dev

Therefore, a successful frontend installation should primarily be verified by confirming that:

1. `npm.cmd ci` completes without installation errors;

2. `npm.cmd run build` completes successfully;
3. `npm.cmd run dev` starts the application on port 5173;
4. the NightOUT login page opens in the browser;
5. the frontend can communicate with the running backend.

3. Dependencies

NightOUT is composed of two main software components: a Spring Boot backend and a React frontend. Each component manages its dependencies through a dedicated configuration file.

The backend dependencies are defined in:

IMPLEMENTATION/backend/pom.xml

The frontend dependencies are defined in:

IMPLEMENTATION/frontend/package.json

The exact frontend dependency versions installed by npm are recorded in:

IMPLEMENTATION/frontend/package-lock.json

Backend Dependencies

The backend is implemented using Java 21 and Spring Boot. Maven manages the required libraries and automatically downloads them when the Maven Wrapper is executed.

The main backend dependencies are listed below.

Dependency	Purpose
Spring Boot Web MVC	Provides the embedded web server, REST controllers, HTTP request handling, and JSON serialization.
Spring Data JPA	Provides database access through JPA entities and repository interfaces.

Spring Validation	Validates incoming API requests using Jakarta Bean Validation annotations.
H2 Database	Provides the in-memory relational database used by the application.
H2 Console	Provides a browser-based interface for inspecting the local development database.
Lombok	Reduces repetitive Java code by automatically generating methods such as getters, setters, and constructors.
Spring Boot Test dependencies	Provide utilities for unit, persistence, validation, and REST API testing.

The project uses the Spring Boot Maven plugin to compile, test, package, and run the backend.

The Maven Wrapper included in the repository ensures that the correct Maven version is used. Therefore, users do not need to install Maven separately.

Frontend Dependencies

The frontend is implemented using React, TypeScript, and Vite. npm manages the frontend dependencies.

The main frontend dependencies and development tools are listed below.

Dependency or tool	Purpose
React	Provides the component-based framework used to build the graphical user interface.
React DOM	Renders the React application in the browser.

React Router DOM	Manages navigation, protected routes, and role-specific pages.
Axios	Sends HTTP requests from the frontend to the Spring Boot REST API.
TypeScript	Adds static type checking to the frontend source code.
Vite	Provides the development server and generates production builds.
React plugin for Vite	Integrates React compilation and development refresh features with Vite.
ESLint	Performs static code-quality checks on JavaScript and TypeScript files.
React and Node type packages	Provide TypeScript declarations for React, React DOM, Node.js, and the Vite configuration.

The frontend does not currently use an external UI component framework. Its interface is implemented through custom React components and styles.

The project also does not include a dedicated frontend automated test framework. The available npm scripts support development, production build generation, linting, and local preview.

Dependency Installation

Backend dependencies are installed automatically when one of the following Maven Wrapper commands is executed:

```
.\mvnw.cmd spring-boot:run
```

or, when using the Git Bash workaround:

```
& "$env:ProgramFiles\Git\bin\bash.exe" -lc './mvnw spring-boot:run'
```

Frontend dependencies are installed using:

npm.cmd ci

The `npm ci` command reads `package-lock.json` and installs the exact dependency versions recorded in the repository.

An internet connection is required the first time Maven and npm download their respective dependencies. After the required packages have been cached locally, later executions may not require downloading them again.

Users should avoid manually changing dependency versions unless they intentionally want to update the project configuration. Changes to `pom.xml`, `package.json`, or `package-lock.json` may affect compatibility and should be tested before being committed to the repository.

4.5 Correct Use of the Application

This section describes how to use the main NightOUT functionalities after the backend and frontend have been started successfully.

The application provides three different roles:

- Normal User;
- Venue Manager;
- PR Manager.

Each role has a dedicated interface, navigation structure, and set of available functionalities.

4.5.1 Sign In, Session, and Role Boundaries

Open the NightOUT application at:

http://localhost:5173

The login page requires an email address and password.

Use one of the predefined demo accounts:

Role	Email	Password	Landing page

Normal User	`daniele.lorenzano@nightout.demo`	`user123`	`/feed`
Venue Manager	`matteo.conti@nightout.demo`	`venue123`	`/manager`
PR Manager	`filippo.scaranello@nightout.demo`	`pr123`	`/pr`

After a successful login, the application redirects the user to the homepage associated with the selected role.

A Normal User is redirected to the event feed, a Venue Manager to the management dashboard, and a PR Manager to the promotional dashboard.

Each role has access only to its dedicated navigation pages. If a user manually opens a protected URL associated with another role, the frontend redirects them to their own role homepage.

The available navigation sections are:

Role	Main navigation
Normal User	Home, Tickets, Pregame, Account
Venue Manager	Dashboard, Events, Tickets, Promotions, Profile
PR Manager	Dashboard, Code / Tickets, Account

The current session is stored in the browser using `localStorage`. This allows the application to restore the signed-in user when the page is refreshed.

The browser stores session-related information but does not store the password used during login.

Log Out

To log out:

1. select the circular avatar or initial displayed in the top-right corner;
2. select the logout action;
3. wait for the application to return to the login page.

Logging out removes the local browser session. Protected pages can no longer be accessed until a new login is completed.

Current Authentication Limitations

The current version does not provide:

- user registration;
- password reset;
- password modification;
- email verification;
- secure server-side sessions;
- multi-factor authentication.

The available credentials are public demo credentials and must only be used for local testing and demonstration.

The role-based frontend routing improves the application flow, but it must not be considered a production security mechanism.